Twin Prime Conditional Endpoint by Finite Contradiction

alegator (coideated with GPT 5.4, 5.5)

May 6, 2026

Abstract

We formalize the Lean endpoint for a finite-contradiction approach to twin prime infinitude. The checked core proves that supposing the finiteness of twins forces a cofinal tail of midpoint-exceptional primes, and that any cofinal tail contradicts a moving-window MP/PM event-pressure certificate. The remaining mathematical content is the pressure certificate itself: fresh eligible exceptional starts must create enough distinct composite-prime and prime-composite middle-pair events in the finite moving window. The generated routed-chain shards provide audit data for this pressure picture, but the generic fresh-start and descent theorems alone do not yet prove fresh event pressure.

1 Introduction

This proof is written and formalized in Lean4 by a "citizen mathematician" with the coideation of GPT 5.4 and 5.5 through web client and Codex interfaces. The twin prime conjecture is the 2-gap case of de Polignac's 1849 conjecture on even prime gaps [1]. Brun proved in 1919 that the reciprocal series over twin primes converges, introducing sieve methods into the problem [2]. Chen proved that infinitely many primes p have $p + 2$ either prime or the product of at most two primes [3]. The modern bounded-gap breakthrough began with Zhang's finite bound for gaps between consecutive primes [4], was sharpened by the Maynard–Tao method [5], and was pushed by Polymath8 to the unconditional bound $H_1 \leq 246$ [6].

This paper does not try to improve sieve weights or follow the bounded-gap $b \leq 246$ route. It uses midpoint rows and recursive MP/PM event descent. If twins were finite, a cofinal tail of exceptional primes would exist. Lean checks that a suitable event-pressure theorem for that tail rules out the tail, and then gives arbitrarily large twin primes. The present repository should be read as a precise conditional endpoint plus finite audit data, not as a closed unconditional proof.

Contents

2 Overview

2.1 Midpoint rows and exceptional primes

The proof endpoint studied here is organized around a very small object. For a prime r, inspect the midpoint row

$$M_r = \{rh : 1 \le h < r\}.$$

The midpoint rh gives the candidate twin pair

$$rh - 1, \qquad rh + 1.$$

If some h in the row gives two primes, then we have a twin-prime pair. A prime r is called an exception, or a midpoint-exceptional prime, if no such index exists.

The project uses this exception object to turn twin infinitude into a first-exception-after-a-bound problem. If twin primes were eventually absent, then for every sufficiently large prime r, all row midpoints rh would lie beyond the last twin midpoint. Then every sufficiently large prime would be exceptional. A cofinal tail of exceptional primes has a first exceptional prime after any chosen finite verified bound. To prove infinitely many twins, it is enough to rule out such a cofinal exceptional tail.

Empirically, the last small exception in the larger search was 127, and it was tested that the next gap is at least

$$V_0 = 191264$$

2.2 Split primes and recursive MP/PM closure

In this certificate, a split prime is a prime for which 5 is a quadratic residue modulo that prime:

$$\exists s, \qquad s^2 \equiv 5 \pmod{p}.$$

The row congruences expose this residue directly. For left rows,

$$4\big(u(u+3)+1\big) = (2u+3)^2 - 5,$$

so $p \mid u(u+3)+1$ forces

$$(2u+3)^2 \equiv 5 \pmod{p}.$$

For right rows,

$$4\big((u+1)(u+4)+1\big) = (2u+5)^2 - 5,$$

so $p \mid (u+1)(u+4)+1$ forces

$$(2u+5)^2 \equiv 5 \pmod{p}.$$

The labels MP and PM denote the two side-labeled finite event classes in the generated middle-pair block: one side records composite-prime middle-pair events and the other records prime-composite middle-pair events.

The recursive MP/PM event closure is the finite descent operation used by the generated certificate. For a split prime p, the program constructs a direct finite target-event set $T(p)$, a finite descended prime set $D(p)$ consisting only of smaller split primes, and the recursively reachable event set

$$R(p) = T(p) \cup \bigcup_{q \in D(p)} R(q).$$

The inequality $q < p$ makes this a well-founded finite recursion. The core certificate implication is that a cofinal exceptional tail makes every event in the computed closure actual in the finite MP/PM block.

The row formulas used by this closure are the two width-two quadratic identities. A left row is an integer u for which

$$p \mid u(u+3) + 1.$$

With

$$h = \frac{u(u+3)+1}{p},$$

one has

$$ph - 1 = u(u+3), \qquad ph + 1 = (u+1)(u+2),$$

using

$$u(u+3) + 2 = (u+1)(u+2).$$

A right row is an integer u for which

$$p \mid (u+1)(u+4) + 1.$$

With

$$h = \frac{(u+1)(u+4)+1}{p},$$

one has

$$ph - 1 = (u+1)(u+4), \qquad ph + 1 = (u+2)(u+3),$$

using

$$(u+1)(u+4) + 2 = (u+2)(u+3).$$

The discriminant of both row congruences is 5, which is why the certificate uses split primes.

The five adjacent slots

$$u,\ u+1,\ u+2,\ u+3,\ u+4$$

are then factored. Any split prime factor $q < p$ in those slots is a descended prime route. Recursion continues from q. The generated computation observes that descent into the early finite MP/PM block is overwhelmingly the common case; the recursive closure formalizes this by exhaustively following all smaller descended primes until the decreasing graph bottoms out. The following table is one concrete route printed by

`tools/print_routing_example.cpp`.

stage	prime	row	quadratic data	next event
1	191281	$R,\ u = 183108,\ h = 175289$	$191281 \cdot 175289 = (183109)(183112) + 1$	$u + 2 = 183110 = 10 \cdot 18311$
2	18311	$R,\ u = 1486,\ h = 121$	$18311 \cdot 121 = (1487)(1490) + 1$	$u + 3 = 1489$
3	1489	$L,\ u = 807,\ h = 439$	$1489 \cdot 439 = 807 \cdot 810 + 1$	$u + 4 = 811$
4	811	$R,\ u = 780,\ h = 755$	$811 \cdot 755 = 781 \cdot 784 + 1$	$u + 1 = 781 = 11 \cdot 71$
5	71	$R,\ u_0 = 6,\ h = 1$	$71 = (7)(10) + 1$	$u_t = 1426 \equiv 6 \pmod{71},\ u_t \in \mathrm{MP} \cap \mathrm{PM}$

This is a typical closure move: each row produces a smaller split prime through one of its five slots, and the final descended row reaches the early finite MP/PM block. Distinct routes can collide at the same side-labeled target event, and some routes can escape the target block or fail to produce a descended split prime before bottoming out. The certificate is built to handle both effects: it counts only unique predicted side-labeled MP/PM pairs in the target block, so duplicate routes do not inflate E_r, and escaping routes are simply absent from the predicted target-event set.

The routing implication used by the proof is exactly this: under a cofinal tail of exceptional primes, every predicted MP/PM pair reached by the recursive descent is forced to be an actual MP/PM pair in the target block. The finite contradiction comes from producing more unique predicted target pairs than the target block actually contains.

2.3 Finite contradiction overview

The generated finite contradiction certificate starts at the first split prime

$$R_0 = 191281.$$

Let

$$E_r = 95569$$

be the number of distinct predicted side-labeled MP/PM events produced by the recursive finite computation, and let

$$E_a = 95568$$

be the size of the actual generated MP/PM event set. Since $E_r > E_a$, a cofinal exceptional tail realizing those predicted events is impossible. The generated C++ certificate emits the arithmetic route chains; the semantic realization step is represented in Lean by the explicit certificate type

`RoutedChainRealizationCertificate`.

Given such a certificate, Lean derives

`GeneratedTailInductionCertificate.no_cofinalExceptionTail_of_routedChainRealization`

from the bridge and the checked strict inequality $E_a < E_r$.

The proof in Lean is intentionally transparent. Everything after this one route-realization certificate is ordinary Lean proof. The routed-chain endpoint is

`GeneratedTailInductionCertificate.arbitrarily_large_twins_of_routedChainRealization`.

Given a value of `RoutedChainRealizationCertificate`, its statement expands to:

$$\forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

2.4 What is generated and what is Lean-checked

The final theorem depends on the standard Lean axioms

`propext`, `Classical.choice`, `Quot.sound`,

and on exactly one project-specific routed-chain realization certificate:

`RoutedChainRealizationCertificate`.

The theorem

`GeneratedTailInductionCertificate.no_cofinalExceptionTail_of_routedChainRealization`

is Lean-derived from

`no_cofinalExceptionTail_of_routedMPPMChainCertificate`.

The larger observed no-exception gap is not a dependency of the final Lean endpoint. It remains useful motivation for why this finite contradiction was searched for at the threshold used by the generated routed-chain certificate.

2.5 Formal object directory for Section 1

`TwinPrimeCertificate/Core.lean`

- `lastObservedException`: the last observed exceptional prime, 127.
- `observedVerifiedTo`: the larger motivating verified bound, 10^9.
- `certificateVerifiedTo`: the reduced certificate threshold, 191264.
- `certificateFirstSplitPrime`: the first split prime used by the recursive certificate, 191281.
- `generatedMPPMCard`: the generated actual MP/PM event count, 95568.
- `GeneratedRoutedMPPMChains.checkedChainCount`: the generated predicted event count, 95569.
- `GeneratedRoutedMPPMChains.checkedChainCount_eq`: evaluates the sharded generated count.
- `ObservedExceptionGap`: the record for a last-exception bound and a later verified bound.
- `ObservedExceptionGap.size`: the numerical size of such a gap.
- `observedExceptionGap`: the 127 to 10^9 motivating gap record.
- `certificateExceptionGap`: the 127 to 191264 certificate gap record.
- `certificateVerifiedTo_le_observed`: compares the reduced certificate gap with the larger observed gap.

`TwinPrimeCertificate/RecursiveMPPMCertificate.lean`

- `left_row_residue5_identity`: the left-row residue-5 identity.
- `right_row_residue5_identity`: the right-row residue-5 identity.

- `quad_route_left_identity`: the left width-two routing identity.
- `quad_route_right_identity`: the right width-two routing identity.
- `encodeSideEvent`: encodes a target coordinate and side label as one natural number.
- `encodeSideEvent_zero_ne_one`: separates the two side labels at a fixed coordinate.
- `encodeSideEvent_injective_of_side_lt_two`: proves injectivity for side labels $0, 1$.
- `DescendedPrimeEdge`: the decreasing-edge predicate for descended primes.
- `DescendedPrimeEdge.decreases`: extracts the strict decrease from a descended edge.
- `DescendedPrimeEdge.irrefl`: rules out a self-edge.
- `DescendedPrimeEdge.no_two_cycle`: rules out a two-cycle.

`TwinPrimeCertificate/RoutedMPPMChainBridge.lean`

- `routed_checkedChainCount_exceeds_generatedMPPMCard`: checks $95568 < 95569$.
- `routedChainPredicted_card_exceeds_actual`: checks the generated overflow inequality.
- `RoutedChainRealizationCertificate`: the explicit semantic realization certificate consumed by the final endpoint.
- `no_cofinalExceptionTail_of_routedMPPMChainCertificate`: the finite-set contradiction from that certificate.

`TwinPrimeCertificate/GeneratedTailInductionCertificate.lean`

- `fromRoutedChainRealization`: converts the routed-chain realization certificate into the tail-induction interface.
- `no_cofinalExceptionTail_of_routedChainRealization`: derives no cofinal exceptional tail.
- `arbitrarily_large_twins_of_routedChainRealization`: the routed-chain twin-prime endpoint.

`TwinPrimeCertificate/Final.lean`

- `arbitrarily_large_twins`: converts any midpoint tail-induction certificate into arbitrarily large twins.

3 Midpoint-exceptional primes

Definition 3.1 (Row index)**.** *For $r, h \in \mathbb{N}$, define*

$$\mathrm{MidpointRowIndex}(r, h) \quad :\Longleftrightarrow \quad 1 \le h \ \wedge \ h < r.$$

This is Lean definition

MidpointRowIndex.

Definition 3.2 (Midpoint and twin witness)**.** *For $r, h \in \mathbb{N}$, define the midpoint*

$$\mathrm{MidpointRowMidpoint}(r, h) = rh.$$

Define

$$\mathrm{MidpointTwinWitnessAt}(r, h) \quad :\Longleftrightarrow \quad \mathrm{Prime}(rh - 1) \ \wedge \ \mathrm{Prime}(rh + 1).$$

These are Lean definitions

MidpointRowMidpoint, *MidpointTwinWitnessAt.*

Definition 3.3 (Midpoint-exceptional prime)**.** *A prime r is a midpoint-exceptional prime, or simply an exception, if*

$$\mathrm{Prime}(r) \quad \textit{and} \quad \forall h, \ (1 \le h < r) \Rightarrow \neg\, \mathrm{MidpointTwinWitnessAt}(r, h).$$

In Lean this is

MidpointExceptionalPrime.

This direct definition says that the midpoint row associated to r contains no twin-prime pair. Earlier searches also used divisor formulations of the same condition, but the reduced Lean endpoint works directly with the midpoint-row predicate above.

3.1 Formal objects used in Section 2

`TwinPrimeCertificate/Core.lean`

- `MidpointRowIndex`: the row-index predicate $1 \le h < r$.
- `MidpointRowMidpoint`: the midpoint rh.
- `MidpointTwinWitnessAt`: the predicate that $rh - 1$ and $rh + 1$ are both prime.
- `MidpointExceptionalPrime`: the definition of a midpoint-exceptional prime.

4 From finite twins to an exceptional tail

Definition 4.1 (Bounded twin midpoints)**.** *The predicate* BoundedTwinMids *says that there exists a bound M such that no midpoint $m > M$ has both $m-1$ and $m+1$ prime:*

$$\exists M,\ \forall m > M,\ \neg(\mathrm{Prime}(m-1) \wedge \mathrm{Prime}(m+1)).$$

This is Lean definition

BoundedTwinMids.

Definition 4.2 (Arbitrarily large twins)**.** *The target theorem is*

$$\mathrm{ArbitrarilyLargeTwins} \quad :\Longleftrightarrow \quad \forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

This is Lean definition

ArbitrarilyLargeTwins.

Lemma 4.3 (Bounded midpoints force a cofinal exceptional tail)**.** *If* BoundedTwinMids *holds, then there exists B such that every prime $r > B$ is a* MidpointExceptionalPrime*.*

Proof. Choose M witnessing BoundedTwinMids. Set $B = M$. Let r be prime and suppose $M < r$. To prove that r is exceptional, let h satisfy $1 \le h < r$. The corresponding midpoint is

$$m = rh.$$

Since $h \ge 1$,

$$r = r \cdot 1 \le rh = m.$$

So $M < r \le m$, so $M < m$. By the defining property of M, the pair $(m-1, m+1)$ is not a twin-prime pair. So no row index h gives a twin pair, so r is a MidpointExceptionalPrime. □

This is Lean theorem

`boundedTwinMids_forces_cofinalTail_midpointExceptionalPrime`.

4.1 Formal objects used in Section 3

`TwinPrimeCertificate/Core.lean`

- `BoundedTwinMids`: the bounded-twin-midpoint hypothesis.
- `ArbitrarilyLargeTwins`: the target infinitude predicate.
- `CofinalExceptionTail`: packages that every prime past a bound is exceptional.
- `boundedTwinMids_forces_cofinalTail_midpointExceptionalPrime`: proves that bounded twin midpoints force a cofinal exceptional tail.
- `arbitrarily_large_twins_of_not_boundedTwinMids`: proves the elementary endpoint from the negation of bounded twin midpoints.

5 First exceptions after a bound

Definition 5.1 (Observed exception gap)**.** *An* ObservedExceptionGap *is a record containing a last exception bound and a later verified bound:*

$$(\text{lastException}, \text{verifiedTo}), \qquad \text{lastException} < \text{verifiedTo}\,.$$

The certificate gap in this project is

$$\text{lastException} = 127, \qquad \text{verifiedTo} = 191264.$$

In Lean:

certificateExceptionGap.

Definition 5.2 (First exception after a gap)**.** *For an exception predicate* $E : \mathbb{N} \to \text{Prop}$ *and a gap* γ*, a value*

$$\text{FirstExceptionAfterGap}(E, \gamma)$$

consists of a prime r *such that:*

(i) $r > \gamma.\,\text{verifiedTo}$*;*

(ii) $E(r)$*;*

(iii) no prime s *with*

$$\gamma.\,\text{verifiedTo} < s < r$$

satisfies $E(s)$*.*

This is Lean structure

FirstExceptionAfterGap.

Lemma 5.3 (A cofinal exceptional tail gives a first exception after any gap)**.** *Let* $E : \mathbb{N} \to \text{Prop}$*. If there exists* B *such that every prime* $r > B$ *satisfies* $E(r)$*, then for every observed gap* γ*, there exists a* $\text{FirstExceptionAfterGap}(E, \gamma)$*.*

Proof. Choose a prime $r_0 > \max(B, \gamma.\,\text{verifiedTo})$, using infinitude of primes. Then $E(r_0)$, so the set

$$S = \{p : \ p \text{ prime}, \ \gamma.\,\text{verifiedTo} < p, \ E(p)\}$$

is nonempty. By well-ordering of $\mathbb{N}$, let R be its least element. Then R is prime, $R > \gamma.\,\text{verifiedTo}$, and $E(R)$. By minimality, no smaller prime s above $\gamma.\,\text{verifiedTo}$ satisfies $E(s)$. So R is the first exceptional prime after γ's verified bound. □

This is Lean theorem

`cofinalTail_gives_firstExceptionAfterGap`.

Lemma 5.4 (Gap monotonicity)**.** *If* $\gamma_1.\,\text{verifiedTo} \le \gamma_2.\,\text{verifiedTo}$*, then any first exception after* γ_2 *induces a first exception after* γ_1*.*

Proof. Let R be the first exception after γ_2. Then R is an exception prime above $\gamma_2.\,\mathrm{verifiedTo}$, so also above $\gamma_1.\,\mathrm{verifiedTo}$. Take the least exception prime above $\gamma_1.\,\mathrm{verifiedTo}$. This least prime is the required first exception after γ_1. □

This is Lean theorem

`firstException_mono_backward`.

The contrapositive, used for forwarding no-first-exception results to later gaps, is

`no_firstException_mono_forward`.

5.1 Formal objects used in Section 4

`TwinPrimeCertificate/Core.lean`

- `ObservedExceptionGap`: the gap record.
- `ObservedExceptionGap.size`: the numerical size of a gap.
- `lastObservedException`: the last observed exceptional prime.
- `certificateVerifiedTo`: the reduced certificate threshold.
- `observedVerifiedTo`: the larger motivating verified threshold.
- `certificateExceptionGap`: the concrete reduced certificate gap.
- `observedExceptionGap`: the concrete motivating observed gap.
- `certificateVerifiedTo_le_observed`: compares the certificate gap with the larger observed gap.
- `FirstExceptionAfterGap`: the first-exception-after-a-gap record.
- `FirstExceptionAfterLastObserved`: the version used by the auxiliary gap checker.
- `ExceptionFreeUpTo`: the finite no-exception predicate.
- `firstExceptionAfterLast_occurs_after_of_exceptionFreeUpTo`: pushes a first exception past a checked window.
- `firstExceptionAfterLast_occurs_after_certificateThreshold_of_exceptionFreeUpTo`: the certificate-threshold specialization.
- `cofinalTail_gives_firstExceptionAfterGap`: extracts a first exception after a chosen gap from a cofinal exceptional tail.
- `firstException_mono_backward`: first-exception monotonicity toward earlier gaps.
- `no_firstException_mono_forward`: the no-first-exception contrapositive toward later gaps.

6 The generated finite certificate

The main generated finite certificate is expressed in Lean as the route-realization bridge

- `RoutedChainRealizationCertificate`

whose mathematical shape is

$$\big(\exists B, \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B)\big) \Rightarrow \mathrm{predictedEvents} \subseteq \mathrm{actualEvents}\,.$$

Lean combines this bridge with the finite event counts to prove:

- `GeneratedTailInductionCertificate.no_cofinalExceptionTail_of_routedChainRealization`

whose mathematical shape is

$$\text{Not } \exists B,\ \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B).$$

The generated file records the numerical overflow:

$$E_a = 95568,$$

$$E_r = 95569,$$

and Lean checks

$$E_a < E_r.$$

The Lean theorem name for the arithmetic inequality is

`routedChainPredicted_card_exceeds_actual`.

Certificate 6.1 (Generated finite contradiction obstruction)**.** *There is no cofinal tail of midpoint-exceptional primes:*

$$\text{Not } \exists B,\ \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B).$$

Generated certificate proof. The C++ program `generate_routed_mppm_chain_certificate.cpp` generates explicit recursive MP/PM route-chain witnesses from the generated MP/PM block. At start split prime 191281, the generated reachable closure contains 95569 distinct side-labeled predicted MP/PM events. The generated finite MP/PM event set contains only 95568 events. Under a cofinal exceptional tail, the route-realization certificate makes every predicted event actual in the side-labeled MP/PM event set. This would inject a set of cardinality 95569 into a set of cardinality 95568, impossible.

The correctness argument for the finite computation is included below in this section. In Lean, the route-realization part is the explicit certificate

`RoutedChainRealizationCertificate`.

The cardinal contradiction from this bridge is checked by

`no_cofinalExceptionTail_of_routedMPPMChainCertificate`.

□

6.1 Correctness of the recursive MP/PM event certificate

This subsection describes the correctness of

`tools/generate_routed_mppm_chain_certificate.cpp`.

The purpose of the algorithm is finite and exact: emit explicit recursive quadratic-routing chains into a generated side-labeled MP/PM event universe. Lean checks the emitted row equations, decreasing descended-prime edges, terminal event encodings, shard ordering, and the count $95569 > 95568$.

6.1.1 Input certificate

The input JSON file

`certificates/generated_mppm_pressure_certificate.json`

contains three finite arrays:

$$\text{Base}, \qquad \text{MP}, \qquad \text{PM}.$$

The algorithm builds a side-labeled predicted event universe

$$U = \{(u, 0) : u \in \text{Base}\} \cup \{(u, 1) : u \in \text{Base}\},$$

encoded by

$$\text{encode}(u, s) = 2u + s.$$

Here $s = 0$ is the MP-side label and $s = 1$ is the PM-side label. The encoding is just parity bookkeeping: the side label is recovered from the parity of $2u + s$, and the coordinate u is recovered by integer division by 2. In Lean this is the function

`encodeSideEvent`.

The theorems

`encodeSideEvent_zero_ne_one`

and

`encodeSideEvent_injective_of_side_lt_two`

record that the two side labels for a fixed u are distinct and that the encoding is injective when $s < 2$. It builds the actual side-labeled MP/PM set

$$A = \{(u, 0) : u \in \text{MP}\} \cup \{(u, 1) : u \in \text{PM}\}.$$

The generated data gives

$$|A| = 95568.$$

6.1.2 Split primes and rows

For a prime p, the predicate `leg5(p)` tests whether 5 is a quadratic residue modulo p. If not, the prime has no rows.

If 5 is a quadratic residue modulo p, the function `sqrt5_mod_prime(p)` returns the two square roots of 5 modulo p. This is computed either by the $p \equiv 3 \pmod 4$ formula or by Tonelli-Shanks [10]. Both branches are standard algorithms satisfying:

$$s \in \texttt{sqrt5\_mod\_prime(p)} \quad \Longleftrightarrow \quad s^2 \equiv 5 \pmod p.$$

For each such s, the algorithm builds two row types. The right row solves

$$(u+1)(u+4)+1 \equiv 0 \pmod p,$$

and the left row solves

$$u(u+3)+1 \equiv 0 \pmod p.$$

The formulas in `rows_for(p)` are exactly the quadratic formula for these two congruences. Whenever the computed midpoint is divisible by p, the algorithm sets

$$h = \frac{(u+1)(u+4)+1}{p} \quad \text{or} \quad h = \frac{u(u+3)+1}{p},$$

and retains the row only when

$$1 \le h < p.$$

So every retained row is a valid midpoint-row index for p, and the row records the five adjacent slots

$$u, u+1, u+2, u+3, u+4.$$

Lean checks the residue and row identities used here as

`left_row_residue5_identity`, `right_row_residue5_identity`,
`quad_route_left_identity`, `quad_route_right_identity`.

6.1.3 Target events

The function `target_events_at(p)` maps rows at p into the finite event universe U. If $p < X$, the row coordinate u is lifted through the congruence class

$$u,\ u+p,\ u+2p, \ldots < X.$$

For each lifted u, the side-labeled events $(u, 0)$ and $(u, 1)$ are looked up in U. If present, their integer identifiers are inserted. The result is sorted and deduplicated. So `target_events_at(p)` returns exactly the finite set of generated Base-side events hit directly by rows at p.

6.1.4 Descended prime edges and recursive closure

For a retained row at p, the algorithm factors each of the five slot values $u, u+1, u+2, u+3, u+4$. A prime factor q is accepted as a descended prime if

$$31 \le q < p \quad \text{and} \quad 5 \text{ is a quadratic residue modulo } q.$$

The strict inequality $q < p$ is essential: descended prime edges always go to smaller primes, so the graph is acyclic. The reduced Lean model records this local fact as

- `DescendedPrimeEdge.decreases`
- `DescendedPrimeEdge.irrefl`
- `DescendedPrimeEdge.no_two_cycle`

The full finite graph construction remains part of the generated C++ route audit.

Define the finite recursive closure $R(p)$ by well-founded recursion on prime size:

$$R(p) = T(p) \cup \bigcup_{q \in D(p)} R(q),$$

where $T(p)$ is the direct target-event set and $D(p)$ is the finite descended prime set. The function `reachable_events(p)` implements exactly this recursion, using memoization. Memoization does not alter the mathematical value; it only caches already-computed $R(q)$. Since every descended prime satisfies $q < p$, the recursion terminates.

6.1.5 Primality and factorization subroutines

The C++ certificate uses deterministic 64-bit Miller-Rabin for primality testing and Pollard-Rho for prime factorization.

6.1.6 Route-chain generation correctness

The main generator is invoked with start split prime $R = 191281$. It computes

$$R(R) = \texttt{reachable\_events(R)}$$

and emits one checked chain witness for each event identifier in the finite set

$$P_{\text{seen}}.$$

The generated index sums the shard-local chain counts, so

$$|P_{\text{seen}}|,$$

is checked in Lean as `GeneratedRoutedMPPMChains.checkedChainCount`.

The generated output for the chosen start prime is:

$$\begin{aligned} R &= 191281, \\ |P_{\text{seen}}| &= 95569, \\ |A| &= 95568. \end{aligned}$$

So the finite predicted event set is strictly larger than the actual side-labeled MP/PM event set.

6.1.7 Certificate implication

The mathematical routing theorem certified by the MP/PM event computation is: if a cofinal tail of midpoint-exceptional primes exists, then every event in the computed finite set

$$P_{\text{seen}}$$

is realized in the actual generated MP/PM event set A. So

$$|P_{\text{seen}}| \leq |A|.$$

But the computed values give

$$95569 = |P_{\text{seen}}| > 95568 = |A|,$$

a contradiction. So no such cofinal exceptional tail exists. This proves the Lean theorem

`GeneratedTailInductionCertificate.no_cofinalExceptionTail_of_routedChainRealization`.

The only project-specific routed-chain certificate needed to derive it is

`RoutedChainRealizationCertificate`.

6.2 Formal objects used in Section 5

`TwinPrimeCertificate/Core.lean`

- `generatedMPPMCard`: the actual generated MP/PM event count 95568.
- `CofinalExceptionTail`: the cofinal-tail hypothesis consumed by the route-realization bridge.
- `MidpointExceptionalPrime`: the exception predicate used in that cofinal tail.

`TwinPrimeCertificate/RecursiveMPPMCertificate.lean`

- `left_row_residue5_identity`: the left-row residue-5 identity.
- `right_row_residue5_identity`: the right-row residue-5 identity.
- `quad_route_left_identity`: the left row algebra identity.
- `quad_route_right_identity`: the right row algebra identity.
- `encodeSideEvent`: encodes a coordinate and side label as one natural number.
- `encodeSideEvent_zero_ne_one`: proves the MP and PM side labels differ at a fixed coordinate.

- `encodeSideEvent_injective_of_side_lt_two`: proves injectivity for labels $0, 1$.
- `DescendedPrimeEdge`: the decreasing-edge predicate for descended primes.
- `DescendedPrimeEdge.decreases`: extracts the strict decrease.
- `DescendedPrimeEdge.irrefl`: rules out self-edges.
- `DescendedPrimeEdge.no_two_cycle`: rules out two-cycles.

`TwinPrimeCertificate/RoutedMPPMChainCertificate.lean`

- `RoutedMPPMChainCertificate.rowProduct`: computes the left or right row product.
- `RoutedMPPMChainCertificate.rowValid`: checks a row equation and cone-index bounds.
- `RoutedMPPMChainCertificate.EdgeWitness.valid`: checks a descended-prime edge.
- `RoutedMPPMChainCertificate.DirectEventWitness.valid`: checks a terminal target event.
- `RoutedMPPMChainCertificate.ChainWitness.valid`: checks a full route chain.
- `RoutedMPPMChainCertificate.strictlyIncreasingIds`: checks that generated event identifiers are unique and ordered.

`TwinPrimeCertificate/GeneratedRoutedMPPMChains/Index.lean`

- `GeneratedRoutedMPPMChains.checkedChainCount`: the predicted event count 95569.
- `GeneratedRoutedMPPMChains.checkedChainCount_eq`: identifies the generated chain count.
- `GeneratedRoutedMPPMChains.checkedActualPredictedCount_eq`: checks the audit count of already-actual predicted chains.
- `GeneratedRoutedMPPMChains.checkedFalsePredictedCount_eq`: checks the audit count of false predicted chains.
- `GeneratedRoutedMPPMChains.shard_boundaries_strict`: proves shard event-id ranges are strictly ordered.

`TwinPrimeCertificate/RoutedMPPMChainBridge.lean`

- `routed_checkedChainCount_exceeds_generatedMPPMCard`: checks $95568 < 95569$.
- `routedChainActualEvents`: the finite actual event set represented by count.
- `routedChainPredictedEvents`: the finite predicted event set represented by count.
- `routedChainPredictedEvents_card_eq`: identifies the predicted event-set cardinality with 95569.
- `routedChainActualEvents_card_eq`: identifies the actual event-set cardinality with 95568.
- `routedChainPredicted_card_exceeds_actual`: checks $E_a < E_r$.
- `RoutedChainRealizationCertificate`: the semantic realization certificate.
- `no_cofinalExceptionTail_of_routedMPPMChainCertificate`: packages the cardinal contradiction as a no-tail theorem.

`TwinPrimeCertificate/GeneratedTailInductionCertificate.lean`

- `fromRoutedChainRealization`: converts the routed-chain realization certificate into the tail-induction interface.

– `no_cofinalExceptionTail_of_routedChainRealization`: the derived no-tail theorem.

– `arbitrarily_large_twins_of_routedChainRealization`: the routed-chain twin-prime endpoint.

6.3 Analytic successor-recovery bridge

The routed-chain certificate proves a finite base overflow. To propagate this overflow after shifting a hypothetical cofinal tail start from B to $B+1$, the proof needs a recovery principle: every event lost by removing the first seed must have arbitrarily late eligible split-prime parents.

The row polynomials

$$L(u) = u(u+3)+1, \qquad R(u) = (u+1)(u+4)+1$$

both have discriminant 5. A DFI–Toth style theorem on roots of quadratic congruences to prime moduli gives exactly the required kind of supply: for a fixed finite residue condition on u, roots of $L(u) \equiv 0 \pmod{p}$ or $R(u) \equiv 0 \pmod{p}$ occur for arbitrarily large split primes p, and with u in the legal-height range. This is not plain Dirichlet in arithmetic progressions; Dirichlet controls the residue class of p, while the recovery theorem must control the residue and interval location of a root u modulo p. The relevant analytic source is the theorem of Duke–Friedlander–Iwaniec [7], with Toth's extension [8] and later refinements such as Ngo [9].

The Lean file `TwinPrimeCertificate/QuadraticRootSupply.lean` does not take this theorem as an axiom. It defines the narrow corollary that must be proved from the analytic literature:

`QuadraticRootParentSupply`.

From this corollary Lean proves the full finite recovery step. The key point is simple: if one arbitrarily late parent exists after every bound, then ask once after N and once after the first parent. This gives two distinct eligible parents, which is exactly the multiplicity-two hypothesis used by the shifted-tail recovery theorem.

`TwinPrimeCertificate/QuadraticRootSupply.lean`

– `QuadraticRowSide`: the left/right row label.

– `quadraticRowValue`: evaluates $L(u)$ or $R(u)$.

– `EventParentSupply`: event-level arbitrarily-late parent supply.

– `multiplicity_two_after_of_eventParentSupply`: turns one-parent-after-any-bound into multiplicity two.

– `exists_exact_eligible_recovery_batch_after_shift_of_eventParentSupply`: exact shifted-tail recovery from event parent supply.

– `QuadraticRootParentSupply`: the DFI–Toth-shaped root-supply corollary.

– `QuadraticRootParentSupply.toEventParentSupply`: forgets row data and keeps the event-level supply.

– `exists_exact_eligible_recovery_batch_after_shift_of_quadraticRootSupply`: exact recovery from the quadratic-root supply.

`TwinPrimeCertificate/DescentPressure.lean`

- `exists_exact_eligible_recovery_batch_after_shift`: the recovery theorem consumed by the quadratic-root bridge.

7 Main theorem

Theorem 7.1 (Routed-chain certificate implies arbitrarily large twin primes)**.** *Given a value of* `RoutedChainRealizationCertificate`*,*

$$\forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

Proof. Let $\mathcal{C}$ be the routed-chain realization certificate. Assume, for contradiction, that twin primes are not arbitrarily large. Then there is some N such that no prime $p \ge N$ has $p+2$ prime. This implies BoundedTwinMids: take $M = N + 1$. If

$$\mathrm{Prime}(m-1) \wedge \mathrm{Prime}(m+1)$$

held for some $m > M$, then $p = m - 1$ would satisfy $p \ge N$ and $\mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2)$, contradiction.

By the bounded-midpoint lemma, BoundedTwinMids gives a cofinal tail of MidpointExceptionalPrime primes. This contradicts the generated certificate theorem

`GeneratedTailInductionCertificate.no_cofinalExceptionTail_of_routedChainRealization`.

So twin primes are arbitrarily large. □

The Lean proof of the last implication is theorem

`arbitrarily_large_twins_of_no_cofinalExceptionTail`.

The routed-chain theorem exported by the project is

`GeneratedTailInductionCertificate.arbitrarily_large_twins_of_routedChainRealization`.

7.1 Formal objects used in Section 6

`TwinPrimeCertificate/Core.lean`

- `no_boundedTwinMids_of_no_cofinalExceptionTail`: turns the no-tail theorem into the negation of bounded twin midpoints.
- `arbitrarily_large_twins_of_no_cofinalExceptionTail`: proves the main endpoint from no cofinal exceptional tail.
- `arbitrarily_large_twins_of_not_boundedTwinMids`: the final elementary unboundedness step.
- `no_boundedTwinMids_of_no_certificateFirstException`: fallback endpoint from no first exception after the certificate gap.
- `arbitrarily_large_twins_of_no_certificateFirstException`: fallback infinitude theorem from that first-exception obstruction.
- `no_later_firstException_of_no_certificateFirstException`: pushes a no-first-exception certificate to later gaps.

– `no_observed_firstException_of_no_certificateFirstException`: specializes the previous theorem to the observed gap.

`TwinPrimeCertificate/GeneratedTailInductionCertificate.lean`

– `no_cofinalExceptionTail_of_routedChainRealization`: supplies the no-tail theorem used by the main endpoint.

`TwinPrimeCertificate/Final.lean`

– `no_cofinalExceptionTail`: converts a midpoint tail-induction certificate into a no-tail theorem.

– `not_boundedTwinMids`: derives unbounded midpoint twins.

– `arbitrarily_large_twins`: the final generic endpoint from a midpoint tail-induction certificate.

References

[1] A. de Polignac, *Recherches nouvelles sur les nombres premiers*, Comptes rendus de l'Académie des sciences, 29:397–401, 1849.

[2] V. Brun, *La série dont les termes sont les inverses des nombres premiers jumeaux est convergente ou finie*, Bulletin des Sciences Mathématiques, 43:100–104, 124–128, 1919.

[3] J. R. Chen, *On the representation of a large even integer as the sum of a prime and the product of at most two primes*, Scientia Sinica, 16:157–176, 1973.

[4] Y. Zhang, *Bounded gaps between primes*, Annals of Mathematics, 179(3):1121–1174, 2014.

[5] J. Maynard, *Small gaps between primes*, Annals of Mathematics, 181(1):383–413, 2015.

[6] D. H. J. Polymath, *Variants of the Selberg sieve, and bounded intervals containing many primes*, Research in the Mathematical Sciences, 1:12, 2014.

[7] W. Duke, J. B. Friedlander, and H. Iwaniec, *Equidistribution of roots of a quadratic congruence to prime moduli*, Annals of Mathematics, 141(2):423–441, 1995.

[8] A. Toth, *Roots of quadratic congruences*, International Mathematics Research Notices, 2000(14):719–739, 2000.

[9] H. T. Ngo, *On roots of quadratic congruences*, arXiv:2107.13301, 2021.

[10] D. Shanks, *Five number-theoretic algorithms*, Congressus Numerantium, 7:51–70, 1973.

A Lean audit

The routed-chain endpoint is audited with

`GeneratedTailInductionCertificate.arbitrarily_large_twins_of_routedChainRealization`.

In Lean this is checked with `#printaxioms`. The theorem is parametric in a value of `RoutedChainRealizationCertificate`. Lean reports only the standard axioms used by ordinary classical finite-set reasoning:

- `propext`
- `Classical.choice`
- `Quot.sound`

The semantic routed-chain certificate is an explicit theorem argument, not a hidden project axiom.

A.1 Formal objects used in Appendix A

`TwinPrimeCertificate/GeneratedTailInductionCertificate.lean`

- `fromRoutedChainRealization`: builds the midpoint tail-induction certificate.
- `no_cofinalExceptionTail_of_routedChainRealization`: the no-tail endpoint from the routed-chain realization certificate.
- `arbitrarily_large_twins_of_routedChainRealization`: the theorem inspected by `#printaxioms`.

`TwinPrimeCertificate/RoutedMPPMChainBridge.lean`

- `RoutedChainRealizationCertificate`: the explicit certificate parameter of the audited theorem.
- `no_cofinalExceptionTail_of_routedMPPMChainCertificate`: the finite cardinality contradiction consumed by the endpoint.

B Repository coverage map

This appendix covers every non-build file in this repository. The `.lake` directory, compiled C++ binaries, and LaTeX auxiliary files are build artifacts. The C++ source file in `tools/` carries the finite certificate generator.

B.1 Lean files

- `TwinPrimeCertificate.lean`: the aggregate import file for the Lean library.
- `TwinPrimeCertificate/Core.lean`: defines the numerical constants 127, 10^9, 191264, 191281, and 95568; defines observed gaps, first-exception records, cofinal exception tails, bounded twin midpoints, midpoint rows, midpoint witnesses, and midpoint-exceptional primes; proves the Lean chain from bounded twin midpoints to a cofinal exceptional tail and from no cofinal exceptional tail to arbitrarily large twin primes.
- `TwinPrimeCertificate/RecursiveMPPMCertificate.lean`: formalizes the row identities, side-event encoding, and decreasing descended-prime edge facts used by the routed-chain checker.
- `TwinPrimeCertificate/DescentPressure.lean`: proves the count-level, escape-bound, exact-recovery, and multiplicity-two recovery lemmas used by the shifted-tail pressure program.
- `TwinPrimeCertificate/QuadraticRootSupply.lean`: isolates the DFI–Toth-shaped quadratic-root parent supply and proves that it implies the exact eligible recovery theorem.
- `TwinPrimeCertificate/FiniteSinkAvoidance.lean`: contains the finite-sink, moving-window event-pressure, pair-pressure, terminal-slot, and ancestor-bridge endpoint surfaces.
- `TwinPrimeCertificate/RoutedMPPMChainCertificate.lean`: defines the Boolean checker for generated row witnesses, edge witnesses, direct event witnesses, full chains, strict event-id ordering, and shard-local counts.
- `TwinPrimeCertificate/GeneratedRoutedMPPMChains/ShardNNN.lean`: the generated shard files. Each shard contains explicit route-chain data and native-decision proofs for validity, ordering, and local counts.
- `TwinPrimeCertificate/GeneratedRoutedMPPMChains/Index.lean`: imports all shards and proves the global generated counts, including `GeneratedRoutedMPPMChains.checkedChainCount` = 95569.
- `TwinPrimeCertificate/RoutedMPPMChainBridge.lean`: converts the checked generated chain count into the finite predicted/actual event-set cardinality contradiction, relative to `RoutedChainRealizationCertificate`.
- `TwinPrimeCertificate/TailInductionCertificate.lean`: packages the no-cofinal-tail result into the midpoint tail-induction interface and proves the generic no-tail consequence.

- `TwinPrimeCertificate/Final.lean`: exports the final no-tail, unbounded-midpoint, and arbitrarily-large-twins endpoints from a tail-induction certificate.
- `TwinPrimeCertificate/GeneratedTailInductionCertificate.lean`: connects `RoutedChainRealizationCertificate` to `Final.lean` and exposes the routed-chain arbitrarily-large-twins theorem.
- `TwinPrimeCertificate/UniqueDescentEndpoint.lean`: exposes the unique moving-window descent, injective descent, and moving-window event-pressure theorem surfaces.

B.2 C++ tools and certificates

- `tools/generate_routed_mppm_chain_certificate.cpp`: reads the finite MP/PM input certificate, computes reachable routed-chain witnesses from the chosen start split prime, emits the Lean shard directory, and prints an audit summary.
- `tools/audit_k2_forward.cpp`: exploratory C++ audit tool for sliding-window event recovery, multiplicity, and shifted-tail experiments.
- `certificates/generated_mppm_pressure_certificate.json`: finite generated Base/MP/PM input block consumed by the routed-chain generator.

B.3 Repository and paper support files

- `paper/twin_prime_certificate_endpoint.tex`: this paper.
- `paper/twin_prime_certificate_endpoint.pdf`: rendered PDF output from the TeX source.
- `README.md`: command-oriented repository summary and regeneration notes.
- `research/pressure_overflow_program.md`: running research log for the pressure, recovery, and DFI–Toth root-supply route.
- `lakefile.toml`: Lean package definition for `TwinPrimeCertificate`.
- `lake-manifest.json`: Lake dependency manifest.
- `lean-toolchain`: Lean toolchain pin.
- `.gitignore`: excludes generated build clutter and local artifacts.

C Lean object index

This index lists every top-level declaration in the non-generated Lean files used by the repository, plus the repeated schema for generated shard files. The section-local formal-object lists above explain the main mathematical role of the objects; this appendix is the exhaustive name map for audit and navigation.

C.1 Core definitions and endpoint logic

`TwinPrimeCertificate/Core.lean`

- `lastObservedException`
- `observedVerifiedTo`
- `certificateVerifiedTo`
- `certificateFirstSplitPrime`
- `certificatePrefixStart`
- `certificatePrefixEnd`
- `generatedMPPMCard`
- `ModFiveOnePrime`
- `exists_modFiveOnePrime_gt`
- `exists_modFiveOnePrime_gt_not_mem_finset`
- `not_cofinal_modFiveOnePrimes_subset_finset`
- `ObservedExceptionGap`
- `ObservedExceptionGap.size`
- `observedExceptionGap`
- `certificateExceptionGap`
- `certificateVerifiedTo_le_observed`
- `FirstExceptionAfterGap`
- `FirstExceptionAfterLastObserved`
- `ExceptionFreeUpTo`
- `firstExceptionAfterLast_occurs_after_of_exceptionFreeUpTo`
- `firstExceptionAfterLast_occurs_after_certificateThreshold_of_exceptionFreeUpTo`
- `CofinalExceptionTail`
- `cofinalTail_forces_seed`
- `certificateFirstSplitPrime_prime`
- `cofinalTail_forces_certificateFirstSplitPrime`
- `cofinalTail_has_modFiveOne_exceptional_seed_after`
- `CofinalTailContradicts`
- `cofinalTail_mono_threshold`
- `cofinalTailContradicts_mono_backward`

- ExceptionPrefix
- cofinalTail_of_exceptionPrefix_of_cofinalTail
- cofinalTailContradicts_of_exceptionPrefix
- cofinalTailContradicts_successor_of_finite_prefix
- cofinalTailContradicts_successor_of_exists_finite_prefix
- cofinalTailContradicts_successor_of_succ_exception
- exists_count_exceeds_of_eventual_increment
- exists_new_image_of_card_lt_image
- card_le_mul_of_fiber_bound
- exists_new_image_of_fiber_bound
- exists_fresh_event_of_escape_and_fiber_bounds
- composite_fiber_bound
- cofinalTailContradicts_of_base_and_successor
- no_cofinalTail_of_base_and_successor_contradiction
- BoundedTwinMids
- ArbitrarilyLargeTwins
- MidpointRowIndex
- MidpointRowMidpoint
- MidpointTwinWitnessAt
- MidpointExceptionalPrime
- exists_prime_dvd_lt_of_not_prime_lt_square
- MidpointExceptionalPrime.exists_descending_prime_divisor
- MidpointPrimeDescentEdge
- odd_prime_has_two_descent_edge
- MidpointPrimeDescentPathBelow
- cofinalTail_exists_primeDescentPathBelow
- boundedTwinMids_forces_cofinalTail_midpointExceptionalPrime
- cofinalTail_gives_firstExceptionAfterGap
- firstException_mono_backward
- no_firstException_mono_forward
- arbitrarily_large_twins_of_not_boundedTwinMids
- no_boundedTwinMids_of_no_cofinalExceptionTail
- arbitrarily_large_twins_of_no_cofinalExceptionTail
- no_boundedTwinMids_of_no_certificateFirstException
- arbitrarily_large_twins_of_no_certificateFirstException
- no_later_firstException_of_no_certificateFirstException
- no_observed_firstException_of_no_certificateFirstException

TwinPrimeCertificate/Final.lean

- no_cofinalExceptionTail
- not_boundedTwinMids
- arbitrarily_large_twins

C.2 Recursive MP/PM and descent-pressure files

TwinPrimeCertificate/RecursiveMPPMCertificate.lean

- left_row_residue5_identity
- right_row_residue5_identity
- FiveQuadraticResidueModulo
- left_row_divisor_gives_residue5
- right_row_divisor_gives_residue5
- quad_route_left_identity
- quad_route_right_identity
- encodeSideEvent
- encodeSideEvent_zero_ne_one
- encodeSideEvent_injective_of_side_lt_two
- DescendedPrimeEdge
- DescendedPrimeEdge.decreases
- DescendedPrimeEdge.irrefl
- DescendedPrimeEdge.no_two_cycle

TwinPrimeCertificate/DescentPressure.lean

- FractionalDescentPressureCertificate
- false_of_fractionalDescentPressure
- false_of_generated_fractionalDescentPressure
- EscapeBoundPressureCertificate
- false_of_escapeBoundPressure
- false_of_generated_escapeBoundPressure
- exists_fresh_event_of_no_escape_fiber_bound
- exists_fresh_event_of_arbitrarily_large_no_escape_batches
- card_le_sum_of_fiber_bounds
- not_arbitrarily_large_batches_into_finite_slots
- exists_finite_recovery_batch_of_lostEvents
- oldEvents_subset_shifted_union_recovery
- exists_exact_recovery_batch_after_shift
- exists_shifted_seed_of_multiplicity_two
- oldEvents_subset_shifted_of_removed_events_multiplicity_two
- oldEvents_subset_shifted_union_later_of_multiplicity_two
- exists_later_ancestor_of_positive_multiplicity
- exists_later_ancestor_of_multiplicity_two
- exists_later_eligible_ancestor_of_multiplicity_two
- lostEvents_have_later_eligible_ancestors_of_multiplicity_two

- exists_exact_eligible_recovery_batch_after_shift

TwinPrimeCertificate/QuadraticRootSupply.lean

- QuadraticRowSide
- quadraticRowValue
- quadraticRowValue_left
- quadraticRowValue_right
- EventParentSupply
- multiplicity_two_after_of_eventParentSupply
- exists_exact_eligible_recovery_batch_after_shift_of_eventParentSupply
- QuadraticRootParentSupply
- QuadraticRootParentSupply.toEventParentSupply
- exists_exact_eligible_recovery_batch_after_shift_of_quadraticRootSupply

C.3 Finite-sink and tail-induction surfaces

TwinPrimeCertificate/FiniteSinkAvoidance.lean

- finite_ancestor_set_misses_arbitrarily_large_modFiveOnePrime
- finite_ancestor_set_not_cofinal_for_modFiveOnePrimes
- exists_modFiveOnePrime_batch_after
- FiniteSinkAvoidanceCertificate
- no_cofinalExceptionTail_of_finiteSinkAvoidance
- arbitrarily_large_twins_of_finiteSinkAvoidance
- ResidueFiniteSinkAvoidanceCertificate
- ResidueFiniteSinkAvoidanceCertificate.toFiniteSinkAvoidance
- no_cofinalExceptionTail_of_residueFiniteSinkAvoidance
- arbitrarily_large_twins_of_residueFiniteSinkAvoidance
- TerminalSlotSinkCertificate
- no_cofinalExceptionTail_of_terminalSlotSink
- arbitrarily_large_twins_of_terminalSlotSink
- slotAncestorUnion
- TerminalSlotAncestorBridgeCertificate
- no_cofinalExceptionTail_of_terminalSlotAncestorBridge
- arbitrarily_large_twins_of_terminalSlotAncestorBridge
- movingSlotAncestorUnion
- MovingTerminalSlotAncestorBridgeCertificate
- no_cofinalExceptionTail_of_movingTerminalSlotAncestorBridge
- arbitrarily_large_twins_of_movingTerminalSlotAncestorBridge
- no_cofinalExceptionTail_of_uniqueMovingWindowDescent

- arbitrarily_large_twins_of_uniqueMovingWindowDescent
- no_cofinalExceptionTail_of_injectiveMovingWindowDescent
- arbitrarily_large_twins_of_injectiveMovingWindowDescent
- MovingWindowEventPressureCertificate
- PairEventPressureCertificate
- PairEventPressureCertificate.toMovingWindowEventPressure
- no_cofinalExceptionTail_of_movingWindowEventPressure
- arbitrarily_large_twins_of_movingWindowEventPressure
- no_cofinalExceptionTail_of_pairEventPressure
- arbitrarily_large_twins_of_pairEventPressure

TwinPrimeCertificate/TailInductionCertificate.lean

- TailInductionCertificate
- TailInductionCertificate.of_finitePrefixRecovery
- TailInductionCertificate.of_no_cofinalTail
- TailInductionCertificate.no_cofinalTail
- MidpointTailInductionCertificate
- no_cofinalExceptionTail_of_tailInductionCertificate
- arbitrarily_large_twins_of_tailInductionCertificate

TwinPrimeCertificate/UniqueDescentEndpoint.lean

- fromUniqueMovingWindowDescent
- no_cofinalExceptionTail_of_uniqueMovingWindowDescent
- arbitrarily_large_twins_of_uniqueMovingWindowDescent
- fromInjectiveMovingWindowDescent
- no_cofinalExceptionTail_of_injectiveMovingWindowDescent
- arbitrarily_large_twins_of_injectiveMovingWindowDescent
- fromMovingWindowEventPressure
- no_cofinalExceptionTail_of_movingWindowEventPressure
- arbitrarily_large_twins_of_movingWindowEventPressure

C.4 Generated-tail and routed-chain bridge files

TwinPrimeCertificate/GeneratedTailInductionCertificate.lean

- GeneratedFinitePrefixRecoveryCertificate
- GeneratedSuccessorRecoveryCertificate
- fromGeneratedFinitePrefixRecovery
- fromGeneratedSuccessorRecovery
- no_cofinalExceptionTail_of_generatedFinitePrefixRecovery

- arbitrarily_large_twins_of_generatedFinitePrefixRecovery
- no_cofinalExceptionTail_of_generatedSuccessorRecovery
- arbitrarily_large_twins_of_generatedSuccessorRecovery
- fromRoutedChainRealization
- no_cofinalExceptionTail_of_routedChainRealization
- arbitrarily_large_twins_of_routedChainRealization
- fromModFiveSeedOverflow
- no_cofinalExceptionTail_of_modFiveSeedOverflow
- arbitrarily_large_twins_of_modFiveSeedOverflow
- fromModFiveGoodSeedOverflow
- no_cofinalExceptionTail_of_modFiveGoodSeedOverflow
- arbitrarily_large_twins_of_modFiveGoodSeedOverflow
- fromTerminalSlotSink
- no_cofinalExceptionTail_of_terminalSlotSink
- arbitrarily_large_twins_of_terminalSlotSink
- fromTerminalSlotAncestorBridge
- no_cofinalExceptionTail_of_terminalSlotAncestorBridge
- arbitrarily_large_twins_of_terminalSlotAncestorBridge
- fromMovingTerminalSlotAncestorBridge
- no_cofinalExceptionTail_of_movingTerminalSlotAncestorBridge
- arbitrarily_large_twins_of_movingTerminalSlotAncestorBridge
- fromUniqueMovingWindowDescent
- no_cofinalExceptionTail_of_uniqueMovingWindowDescent
- arbitrarily_large_twins_of_uniqueMovingWindowDescent

TwinPrimeCertificate/RoutedMPPMChainBridge.lean

- routed_checkedChainCount_exceeds_generatedMPPMCard
- routedChainActualEvents
- routedChainActualEvents_card
- routedChainPredictedEvents
- routedChainPredictedEvents_card
- routedChainPredictedEvents_card_eq
- routedChainActualEvents_card_eq
- routedChainPredicted_card_exceeds_actual
- cofinalTail_before_certificatePrefix_forces_generatedSeed
- certificateVerifiedTo_lt_certificatePrefixStart
- cofinalTail_certificateVerifiedTo_forces_generatedSeed
- chain_start_eq_certificateFirst_of_mem_allStartsIn
- chain_start_exceptional_of_cofinalTail_before_prefix

- chain_start_exceptional_of_cofinalTail_certificateVerifiedTo
- RoutedBaseCaseRealizationCertificate
- cofinalTailContradicts_certificateVerifiedTo_of_routedBaseCaseCertificate
- RoutedChainRealizationCertificate
- no_cofinalExceptionTail_of_routedMPPMChainCertificate
- ModFiveSeedOverflowCertificate
- no_cofinalExceptionTail_of_modFiveSeedOverflowCertificate
- ModFiveGoodSeedOverflowCertificate
- no_cofinalExceptionTail_of_modFiveGoodSeedOverflowCertificate

C.5 Routed-chain checker and generated shard schema

TwinPrimeCertificate/RoutedMPPMChainCertificate.lean

- X
- rowProduct
- rowValid
- rowValid_typ_left_or_right
- rowValid_h_pos
- rowValid_h_lt_parent
- rowValid_mul_eq
- slotValue
- EdgeWitness
- EdgeWitness.valid
- EdgeWitness.rowValid_of_valid
- EdgeWitness.slot_le_four_of_valid
- EdgeWitness.child_lt_parent_of_valid
- EdgeWitness.quotient_mul_eq_slotValue_of_valid
- DirectEventWitness
- DirectEventWitness.valid
- DirectEventWitness.rowValid_of_valid
- DirectEventWitness.targetU_eq_of_valid
- DirectEventWitness.targetU_lt_X_of_valid
- DirectEventWitness.side_lt_two_of_valid
- DirectEventWitness.code_eq_of_valid
- ChainWitness
- ChainWitness.lastParentAux
- ChainWitness.lastParent
- ChainWitness.chainConnectedAux
- ChainWitness.valid

- ChainWitness.terminal_valid_of_valid
- ChainWitness.terminal_targetU_lt_X_of_valid
- ChainWitness.connectedAux_of_valid
- ChainWitness.terminal_parent_eq_lastParent_of_valid
- ChainWitness.terminal_code_eq_of_valid
- ChainWitness.terminal_eventId_eq_of_valid
- ChainWitness.actual_zero_or_one_of_valid
- ChainWitness.edge_valid_of_connectedAux_head
- ChainWitness.edge_parent_eq_of_connectedAux_head
- StrictlyIncreasingById
- strictlyIncreasingIdsFrom
- strictlyIncreasingIds
- allValid
- allStartsIn
- chain_valid_of_allValid
- chain_terminal_targetU_lt_X_of_allValid
- actualCount
- falseCount

TwinPrimeCertificate/GeneratedRoutedMPPMChains/Index.lean

- shardCount
- checkedChainCount
- checkedActualPredictedCount
- checkedFalsePredictedCount
- checkedChainCount_eq
- checkedActualPredictedCount_eq
- checkedFalsePredictedCount_eq
- shard_boundaries_strict
- checkedShardsStartInCertificatePrefix

TwinPrimeCertificate/GeneratedRoutedMPPMChains/ShardNNN.lean
Each generated shard has the same declaration schema below, inside its shard namespace. The concrete chain arrays differ by shard.

- chains
- chainCount
- actualPredictedCount
- falsePredictedCount
- firstId
- lastId
- chains_length

- chains_valid
- chains_start_in_certificatePrefix
- chains_strictlyIncreasing
- actualCount_eq
- falseCount_eq